

Base de Datos vs. Esquema en PostgreSQL

Guía didáctica para estudiantes

Caso de referencia: ParkFlow 360

Idea central

En PostgreSQL, una base de datos es un contenedor lógico de alto nivel. Dentro de una base de datos puede existir uno o varios esquemas, y dentro de cada esquema se organizan tablas, vistas, secuencias, funciones y otros objetos.

Objetivo: comprender la jerarquía PostgreSQL y saber ubicar correctamente cada objeto antes de crear las tablas del proyecto.

1. Objetivos de aprendizaje

- Diferenciar claramente una instancia/servidor PostgreSQL, una base de datos, un esquema y una tabla.
- Comprender que una misma base de datos puede contener múltiples esquemas.
- Entender para qué sirve el esquema public y por qué un proyecto puede crear un esquema propio.
- Interpretar nombres calificados como parkflow.usuario o auditoria.evento.
- Comprender qué ocurre en DataGrip al seleccionar una base de datos y visualizar sus esquemas.
- Aplicar estos conceptos antes de ejecutar el script de creación de tablas de ParkFlow 360.

Regla para memorizar

Servidor PostgreSQL → Base de datos → Esquema → Tabla → Filas.

2. La jerarquía de organización en PostgreSQL

PostgreSQL organiza la información en distintos niveles. Cada nivel cumple una responsabilidad diferente. Para comprender correctamente dónde se crean las tablas, conviene visualizar la estructura de arriba hacia abajo.

```
Servidor PostgreSQL
├─ Base de datos: parkflow360
│   ├── Esquema: public
│   ├── Esquema: parkflow
│   │   ├── Tabla: usuario
│   │   ├── Tabla: cliente
│   │   ├── Tabla: reserva
│   │   └─ Tabla: estancia
│   ├── Esquema: auditoria
│   │   └─ Tabla: evento_auditoria
│   └─ Esquema: reportes
│       └─ Vista: ocupacion_diaria
```

La imagen mental correcta es que la base de datos contiene esquemas. Un esquema NO reemplaza a una base de datos y una tabla NO existe “suelta”: siempre pertenece a un esquema dentro de una base de datos.

3. ¿Qué es una base de datos?

Una base de datos es un contenedor lógico de alto nivel dentro de una instancia PostgreSQL. Normalmente representa un sistema, aplicación, ambiente o conjunto de información que se desea mantener separado de otros.

```
CREATE DATABASE parkflow360;
```

En un mismo servidor PostgreSQL pueden existir varias bases de datos, por ejemplo:

```
Servidor PostgreSQL
├─ postgres
├─ parkflow360
└─ peoplecole
```

```
| autolink
| techmarket
```

- Cada conexión PostgreSQL trabaja dentro de una base de datos concreta.
- La base de datos tiene sus propios esquemas y objetos.
- Dos bases de datos diferentes están más aisladas entre sí que dos esquemas de una misma base.
- En DataGrip se especifica la base de datos al configurar el Data Source.

Importante

Cuando te conectas a parkflow360, trabajas dentro de esa base. El objetivo del script de ParkFlow no es crear otra base, sino crear un esquema y sus tablas dentro de parkflow360.

4. ¿Qué es un esquema?

Un esquema (schema) es un espacio de nombres y organización que existe dentro de una base de datos. Sirve para agrupar objetos relacionados y evitar conflictos de nombres.

```
CREATE SCHEMA parkflow;
```

Después de crearlo, las tablas pueden ubicarse dentro del esquema:

```
CREATE TABLE parkflow.usuario (...);
CREATE TABLE parkflow.reserva (...);
```

El nombre completo de una tabla se expresa como:

esquema.tabla

Ejemplo: parkflow.usuario

5. Una base de datos puede tener más de un esquema

Sí. Esta es una de las ideas más importantes de esta guía. Una sola base de datos puede contener muchos esquemas y cada esquema puede organizar objetos con finalidades diferentes.

Base de datos: parkflow360

```
| public
| parkflow
| auditoria
| reportes
| integracion
```

Esquema	Finalidad posible	Ejemplos de objetos
public	Esquema creado habitualmente por defecto	objetos generales o de laboratorio

parkflow	Núcleo transaccional del sistema	usuario, cliente, reserva, estancia
auditoria	Trazabilidad técnica o funcional	evento_auditoria, acceso
reportes	Objetos orientados a consulta	vistas, vistas materializadas
integracion	Objetos de interoperabilidad	staging, colas o tablas de intercambio

Conclusión

No es necesario crear una base de datos distinta para cada grupo de tablas. Si los objetos pertenecen al mismo sistema y deben convivir estrechamente, los esquemas pueden ser una excelente forma de organización interna.

6. El esquema public

Cuando se crea una base de datos PostgreSQL, normalmente existe un esquema llamado public. Si se crea una tabla sin indicar explícitamente el esquema, PostgreSQL puede ubicarla en public dependiendo del search_path de la sesión.

```
CREATE TABLE cliente (...);

-- Puede terminar como:
public.cliente
```

En ParkFlow 360 se utiliza un esquema propio para que las tablas del proyecto queden agrupadas de forma explícita:

```
CREATE SCHEMA IF NOT EXISTS parkflow;
SET search_path TO parkflow, public;
```

El search_path indica en qué esquemas buscar objetos no calificados y, en determinadas operaciones, dónde crear objetos cuando no se escribe el nombre del esquema.

```
SELECT * FROM usuario;

-- Con parkflow primero en search_path, PostgreSQL puede resolverlo como:
SELECT * FROM parkflow.usuario;
```

7. Base de datos y esquema: comparación directa

Aspecto	Base de datos	Esquema
Nivel	Contenedor lógico superior	Subdivisión interna de una base de datos
Ejemplo ParkFlow	parkflow360	parkflow
Creación	CREATE DATABASE	CREATE SCHEMA
Conexión	La conexión selecciona una base concreta	Se usa dentro de la base conectada
Contiene	Esquemas y objetos asociados	Tablas, vistas, secuencias, funciones,

		etc.
Aislamiento	Mayor	Menor; los esquemas conviven en la misma base
Nombre de tabla	No forma parte del nombre SQL usual de una tabla dentro de la sesión	Sí: esquema.tabla
Uso típico	Separar aplicaciones/ambientes/conjuntos de datos	Organizar módulos, dominios o responsabilidades

8. El esquema también funciona como espacio de nombres

Dos esquemas distintos pueden contener tablas con el mismo nombre. PostgreSQL las considera objetos diferentes porque su nombre completo incluye el esquema.

```
parkflow.usuario
auditoria.usuario
legacy.usuario
```

Por ello, estas consultas pueden apuntar a objetos completamente diferentes:

```
SELECT * FROM parkflow.usuario;
SELECT * FROM legacy.usuario;
```

Idea clave

El esquema ayuda a evitar colisiones de nombres y hace explícito el contexto de cada objeto.

9. Aplicación al proyecto ParkFlow 360

Para el laboratorio de ParkFlow 360 trabajamos con una base de datos llamada parkflow360 y un esquema principal llamado parkflow. La estructura conceptual queda así:

```
Servidor PostgreSQL
└─ parkflow360                ← BASE DE DATOS
   └─ public                  ← ESQUEMA por defecto
      └─ parkflow             ← ESQUEMA del proyecto
         ├── usuario           ← TABLA
         ├── rol
         ├── usuario_rol
         ├── estacionamiento
         ├── zona
         ├── espacio
         ├── cliente
         ├── vehiculo
         ├── tarifa
         ├── reserva
         └── estancia
```

```
├─ pago_operativo
├─ bloqueo_espacio
├─ incidencia
├─ historial_estado
└─ auditoria
```

Así, el nombre calificado de la tabla reserva es:

```
parkflow.reserva
```

10. Secuencia correcta antes de crear las tablas

Paso	Acción
Paso 1	Conectarse al servidor PostgreSQL con un usuario autorizado.
Paso 2	Crear el usuario/rol de laboratorio, si todavía no existe.
Paso 3	Crear la base de datos parkflow360 y definir su propietario.
Paso 4	Crear o utilizar un Data Source de DataGrip conectado específicamente a parkflow360.
Paso 5	Dentro de parkflow360, crear el esquema parkflow.
Paso 6	Establecer el search_path o utilizar nombres calificados.
Paso 7	Ejecutar el script de creación de las 16 tablas en el orden de dependencias.
Paso 8	Refrescar el árbol de DataGrip y verificar parkflow → Tables.

11. ¿Cómo se refleja esto en DataGrip?

Al configurar un Data Source PostgreSQL en DataGrip se define, entre otros datos, el servidor, el puerto, el usuario y la base de datos. Si la base indicada es parkflow360, DataGrip trabajará dentro de esa base.

```
Host: localhost
Port: 5432
Database: parkflow360
User: parkflow_admin
```

Después de sincronizar el Data Source, el árbol puede visualizarse conceptualmente así:

```
parkflow360
├─ Schemas
│   ├── public
│   └─ parkflow
│       └─ Tables
│           ├── usuario
│           ├── reserva
│           ├── estancia
│           └─ ...
```

Error común

Crear correctamente la base parkflow360, pero abrir una consola conectada a postgres y ejecutar allí el script. Antes de ejecutar DDL, verifica siempre la base activa.

```
SELECT current_database();  
SELECT current_user;
```

12. Consultas útiles para comprobar base y esquemas

```
-- Base de datos actual  
SELECT current_database();  
  
-- Usuario actual  
SELECT current_user;  
  
-- Esquemas visibles  
SELECT schema_name  
FROM information_schema.schemata  
ORDER BY schema_name;  
  
-- Tablas del esquema parkflow  
SELECT table_schema, table_name  
FROM information_schema.tables  
WHERE table_schema = 'parkflow'  
    AND table_type = 'BASE TABLE'  
ORDER BY table_name;
```

13. ¿Cuándo conviene usar varios esquemas?

No existe una única estrategia válida para todos los proyectos. Sin embargo, varios esquemas pueden ser útiles cuando existe una necesidad clara de organización, seguridad o separación lógica dentro de la misma base.

- Separar módulos o dominios funcionales grandes.
- Aislar objetos de auditoría o trazabilidad.
- Mantener vistas y objetos de reporting separados del núcleo transaccional.
- Separar tablas temporales o de integración.
- Gestionar permisos por grupo de objetos.
- Mantener una estructura clara en sistemas grandes sin crear bases independientes innecesarias.

No exagerar

Tener muchos esquemas no mejora automáticamente un diseño. Deben existir por una razón concreta. Para el laboratorio inicial de ParkFlow 360 basta con un esquema principal parkflow, aunque comprender la posibilidad de múltiples esquemas es fundamental.

14. Errores frecuentes que debes evitar

Error	Corrección conceptual
Confundir base con esquema	parkflow360 es la base; parkflow es el esquema.
Crear que public es una base	public es un esquema dentro de la base conectada.
Ejecutar el script en la base equivocada	Revisa current_database() antes del DDL.
No refrescar DataGrip	Después de crear objetos puede ser necesario Synchronize/Refresh.
Omitir el esquema sin entender search_path	Usa parkflow.tabla o controla el search_path.
Pensar que una base sólo admite un esquema	Puede tener múltiples esquemas simultáneamente.

15. Preguntas de autoevaluación y defensa

1. ¿Cuál es la diferencia fundamental entre una base de datos y un esquema en PostgreSQL?
2. ¿Puede una base de datos tener varios esquemas? Explica con un ejemplo.
3. ¿Qué es el esquema public?
4. ¿Qué significa parkflow.usuario?
5. ¿Por qué podríamos tener parkflow.usuario y legacy.usuario al mismo tiempo?
6. ¿Qué función cumple search_path?
7. ¿Qué comando crea una base de datos? ¿Y cuál crea un esquema?
8. ¿Cómo verificas en SQL qué base de datos está usando tu consola?
9. ¿Dónde deberían aparecer las 16 tablas de ParkFlow después de ejecutar el script?
10. ¿Cuándo preferirías crear un esquema adicional en lugar de otra base de datos?

16. Ejercicio práctico

Realiza el siguiente ejercicio en una base de prueba o con autorización del docente. El objetivo es comprobar físicamente la diferencia entre base de datos, esquema y tabla.

```
-- Ya conectado a una base de prueba:
CREATE SCHEMA ventas;
CREATE SCHEMA soporte;

CREATE TABLE ventas.cliente (
    cliente_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(120) NOT NULL
);

CREATE TABLE soporte.cliente (
    cliente_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(120) NOT NULL
);
```



```
SELECT * FROM ventas.cliente;  
SELECT * FROM soporte.cliente;
```

Observa que existen dos tablas llamadas cliente, pero son distintas porque pertenecen a esquemas diferentes.

17. Resumen final

Debes poder decirlo sin memorizar de forma mecánica

Una instancia PostgreSQL puede contener varias bases de datos. Cada base de datos puede contener varios esquemas. Cada esquema organiza tablas y otros objetos. En ParkFlow 360 nos conectamos a la base parkflow360 y creamos las tablas dentro del esquema parkflow.

```
POSTGRESQL  
└─ BASE DE DATOS: parkflow360  
    └─ ESQUEMA: parkflow  
        └─ TABLA: usuario  
            └─ FILAS: datos de usuarios
```

18. Checklist de dominio

- ☐ Puedo distinguir servidor, base de datos, esquema y tabla.
- ☐ Sé que una base de datos puede tener varios esquemas.
- ☐ Sé qué significa public.
- ☐ Sé interpretar parkflow.usuario.
- ☐ Puedo explicar para qué sirve search_path.
- ☐ Sé verificar current_database() antes de ejecutar un script.
- ☐ Sé dónde buscar las tablas de ParkFlow en DataGrip.